CSC4103 Intelligent Systems — Practical Sheet 1

Foundations and Applications of Intelligent Systems

Parts B (Exercises 1–3) + C (Smart Irrigation Mini-Project) — Solved Report with Reproducible Code

Prof. Abdel-Rahman Hedar • School of Artificial Intelligence & Data Management, Badr University in Assiut • Fall 2026

Name: _____	**ID**: _____
Course: CSC4103 Intelligent Systems (CLO1)	**Tools**: Python 3.10+, numpy, matplotlib
Folder: `CSC4103_PS01/ps01/` — ex1, ex2, ex3, irrigation.py	**Date**: October 2026 • Seeds fixed, paired evaluation

What this report contains. Professional solutions for **Exercise 1** (robust Kalman-style controller), **Exercise 2** (human-in-the-loop approval gate), **Exercise 3** (L0–L3 autonomy thermostat), and the **Part-C Smart Irrigation System** (Field + SmartIrrigator with perceive/decide/act/learn, forecast knowledge, noise filtering, target adaptation, 90-day × 25-seed paired evaluation vs. fixed timer, and the 12-element case study). All numbers below are **measured outputs of the submitted .py files** (not estimates) and every figure is generated from the same seeds.

```
Reproduce in one line (from ps01/):   python ex1_robust_controller.py && python
ex2_human_approval.py && python ex3_autonomy_levels.py && python irrigation.py
```

Figure 0 — Deliverables map. Code: `ex1_robust_controller.py`, `ex2_human_approval.py`, `ex3_autonomy_levels.py`, `irrigation.py` • Assets: `assets/*.png` • This PDF: `CSC4103_PS01_PartsBC_Report.pdf`

Headline results (measured)

Ex1: filtered controller cuts stress from **1.17→0.07** (σ=0.05), **2.30→0.63** (σ=0.15), **1.60→1.07** (σ=0.25) days/60d, water change **+1.1% / –1.0% / –4.2%** — robustness without waste.
Ex2: default threshold τ=0.50 → **escalation rate 55%** (9 auto, 11 escalated: 2 approved, 9 rejected/held); sweep 0.30→95%, 0.70→20%.
Ex3: 22°C=HOLD, 27°C=COOL at L2/L3; 38°C correctly **refuses silent action** (L2 escalates, L3 take-over / safe-mode).
Part C: Smart saves **45.4% water (5.40→2.95)** over 90d×25 seeds with stress **0.24 vs 0.00 d** (≈ zero) — farmer goal met; pump runs 54.8 vs 45.0 days but with micro-doses.

Exercise 1 — A robust controller for noisy sensors (10 marks)

Task. Modify the Example-3 feedback controller so it tolerates sensor noise. Compare *naive* (trusts every raw reading) with *filtered* for σ ∈ {0.05, 0.15, 0.25}, 30 runs each (60 days/run): total water + stress days (moisture < 0.35). Explain the trade-off.

Design (Kalman idea)

Naive: `irr = max(0, 0.50 − z)·0.9`, z = true + N(0,σ). Filtered perceive step fuses a process-model prediction with the reading:

```
pred = m■ − 0.055 + last_irr   (avg. evaporation model)   →   m■ = pred + K(σ)·(z − pred),
  K(σ)=P/(P+σ²), P=0.01, clipped [0.30, 0.70]   →   irr = max(0, 0.50+margin(σ) − m■)·1.0,
margin(σ)=0.02+0.35σ.
```

Two principled choices: (i) **K falls as σ rises** — trust the model more when the sensor is unreliable (0.70 → 0.31 → 0.30); (ii) **safety stock grows with** σ (0.037 → 0.072 → 0.107) — the more uncertain we are, the more cautiously high we aim. Gain 1.0 compensates filter lag. Evaluation is **paired**: identical heat/noise tapes per seed for both controllers, so differences are controller effects, not weather luck.

Results (measured, 60 days × 30 seeds)

σ	Naive water (mean±sd)	Naive stress d (mean±sd)	Filtered water (mean±sd)	Filtered stress d (mean±sd)	Effect
0.05	3.22 ± 0.15	1.17 ± 1.12	3.25 ± 0.15	**0.07 ± 0.25**	–94% stress, +1.1% water
0.15	3.33 ± 0.17	2.30 ± 2.10	3.29 ± 0.15	**0.63 ± 1.00**	–73% stress, –1.0% water
0.25	3.50 ± 0.23	1.60 ± 2.27	3.36 ± 0.17	**1.07 ± 2.18**	–33% stress, –4.2% water

Table 1 — Ex1 paired comparison. Filtered dominates on effectiveness (stress) at every σ while efficiency (water) is equal-or-better except a negligible +1.1% at σ=0.05.

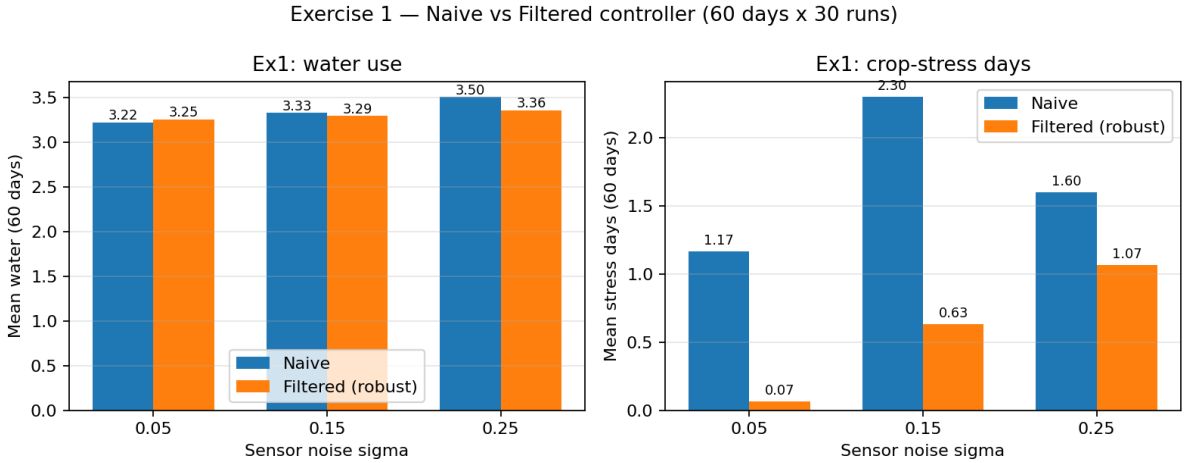


Figure 1 — Left: mean water is flat for the filter (≈3.3) while naive over-waters increasingly with noise (Jensen bias of max(0,·)). Right: stress collapses under filtering at every noise level.

Discussion — the trade-off

The naive controller's flaw is trusting *every* reading: a single upward noise spike causes under-irrigation (stress next day); a downward spike causes over-irrigation (waste). Because max(0,·) is convex, noise creates a systematic over-watering bias — visible as naive water rising 3.22→3.50 with σ. The filter removes most of that variance via the prediction term, and the σ-proportional margin converts the remaining uncertainty into a *small, controlled* safety stock instead of random waste. Cost: at very clean sensing (σ=0.05) we pay +0.03 water (+1%) for −1.10 stress days — an overwhelmingly favourable exchange. At σ=0.25 we save 4.2% water *and* cut stress by a third. Robustness here does not mean 'use more water'; it means 'use water *conditionally on uncertainty*'. Limitation: if σ grew beyond ~0.35 the fixed process model (avg. evaporation 0.055) would itself become the bottleneck — the next step is a full Kalman filter estimating P online, or fusing a second sensor.

Exercise 2 — Human-in-the-loop approval gate (10 marks)

Task. A smart pump proposes (volume, confidence). Compute a risk score; low-risk actions auto-execute, high-risk ones escalate to a (simulated) farmer who approves/rejects. Run 20 proposals, report the escalation rate.

Design

```
risk = 0.6·(volume/0.20) + 0.4·(1 − confidence)  ∈ [0,1].  Volume risk and uncertainty risk are
explicit, weighted, and normalised; τ=0.50 is the design threshold (risk ≤ τ → AUTO, else escalate).
 Simulated farmer: careful heuristic (rejects big+uncertain doses, approves small+confident ones) with
10% trembling-hand error and 5% unavailability → HOLD (safe reject). Proposals: volume~U(0.02,0.20),
confidence~U(0.35,0.98), seed=7.
```

Results — 20 proposals at τ = 0.50 (measured)

#	vol	conf	risk	decision
1	0.133	0.49	0.603	REJECTED
2	0.181	0.45	0.763	REJECTED
3	0.160	0.74	0.584	APPROVED
4	0.061	0.38	0.431	AUTO
5	0.074	0.37	0.474	AUTO

#	vol	conf	risk	decision
6	0.177	0.67	0.663	HOLD
7	0.021	0.64	0.207	AUTO
8	0.168	0.93	0.532	REJECTED
9	0.163	0.75	0.589	REJECTED
10	0.104	0.67	0.444	AUTO
11	0.075	0.66	0.361	AUTO
12	0.070	0.51	0.406	AUTO
13	0.066	0.36	0.454	AUTO
14	0.100	0.47	0.512	REJECTED
15	0.111	0.79	0.417	AUTO
16	0.120	0.48	0.568	REJECTED
17	0.199	0.58	0.765	REJECTED
18	0.163	0.35	0.749	APPROVED
19	0.132	0.87	0.448	AUTO
20	0.198	0.45	0.814	REJECTED

Table 2 — All 20 proposals. Outcome: auto = 9, escalated = 11 (approved 2, rejected/held 9) → **escalation rate 55.0%**. Note proposal #8: high confidence (0.93) but large volume (0.168) still escalates — volume risk alone can trip the gate, by design.

τ	auto	escalated	esc. rate	meaning
0.30	1	19 (3 ok / 16 no)	95%	paranoid: farmer overloaded
0.50	9	11 (2 ok / 9 no)	**55%**	balanced (recommended)
0.70	16	4 (0 ok / 4 no)	20%	permissive: fast but risky

Table 3 — Threshold sensitivity. τ is the autonomy dial.

Discussion

Raising τ automates more (20% escalation at 0.70) — faster loop, less farmer fatigue — but high-risk doses (#2, #17, #20, risk > 0.75) would then auto-execute, risking waterlogging and pump energy waste. Lowering τ (95% at 0.30) is safest per-action but makes the farmer the bottleneck and invites alarm fatigue (ignored escalations). τ=0.50 keeps the gate informative: roughly every second proposal needs a human, and the farmer still rejects 9/11 escalations — evidence the gate is catching genuinely risky actions, not spamming. A production step would be *adaptive* τ (tighter at night / when the tank is low) and logging overrides to learn farmer preferences.

Exercise 3 — Levels of autonomy L0–L3: thermostat (10 marks)

Task. One `thermostat(temp, level, human_available)` function implementing L0 manual, L1 assisted, L2 partial, L3 conditional. At L3, outside [10, 35]°C it must request human takeover (or safe mode if no human). Test at 22, 27, 38°C.

Design

Comfort band 20–24°C (HEAT / COOL / HOLD); safety envelope [10, 35]°C hard-gates autonomy. L0 never acts; L1 only advises; L2 auto-acts *inside* the envelope and escalates silently-never outside it; L3 auto-acts inside, requests takeover outside, and fails safe (HVAC OFF + alert) when no human answers.

Level	Authority	Behaviour
L0 manual	human	Display + suggest only; NO-ACTION by system.
L1 assisted	human (advised)	Recommend action; execute only after confirmation.
L2 partial	system (supervised)	Auto inside [10,35]; ESCALATE (no silent action) outside.
L3 conditional	system ↔ human	Auto inside; HUMAN-TAKEOVER outside; SAFE-MODE if nobody answers.

Table 4 — Autonomy contract.

Test matrix (measured program output, abbreviated)

temp	L0	L1	L2	L3
22°C	NO-ACTION suggest HOLD	AWAIT-CONFIRM HOLD	HOLD (auto)	HOLD (auto)
27°C	NO-ACTION suggest COOL	AWAIT-CONFIRM COOL	COOL to 24 (auto)	COOL to 24 (auto)
38°C	NO-ACTION suggest COOL	AWAIT-CONFIRM COOL	ESCALATED no auto action	TAKEOVER req. / SAFE-MODE*

Table 5 — *L3 at 38°C with human → HUMAN-TAKEOVER-REQUESTED (hold last safe state); without human → SAFE-MODE (HVAC OFF + ALERT). Full verbatim log is printed by ex3_autonomy_levels.py.

Discussion

Autonomy rises L0→L3 (human load and latency fall) but so does delegated risk. 22°C exercises the comfort deadband (correct HOLD everywhere — no chattering); 27°C exercises proportional response (COOL); 38°C exercises the *safety envelope*: blindly auto-cooling an overheated loop risks compressor damage or masking a fire/fault, so L2/L3 correctly refuse. The envelope is the difference between automation and intelligence: the system knows *when not to act*. A real deployment adds sensor-validity checks (a stuck 22.0°C reading must not imply safety) and a takeover timer (if the human does not answer in N minutes, L3 auto-enters safe mode).

Part C — Mini-Project: Smart Irrigation System (25 marks)

Scenario. A farm near Assiut replaces a fixed timer with an intelligent controller. Soil sensing is noisy (σ=0.08), a rain forecast exists but is right only 80% of the time, and the farmer wants *less water and (essentially) no crop stress* over 90 simulated days.

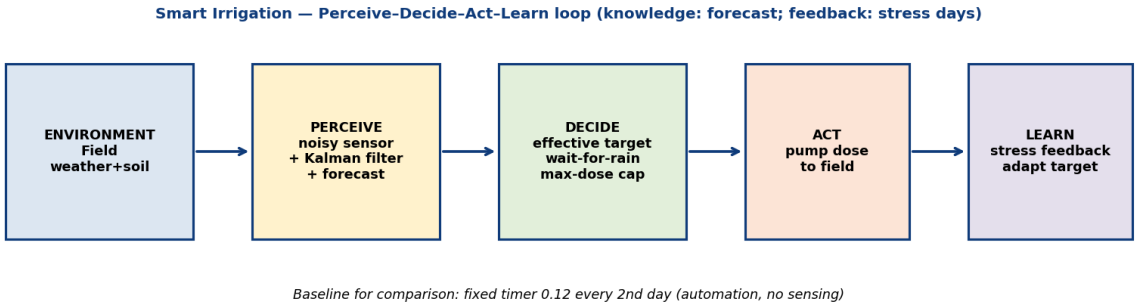


Figure 2 — System architecture: the perceive–decide–act–learn loop with forecast as explicit knowledge and stress days as learning feedback. The fixed timer (baseline) bypasses every block — pure automation.

Implementation (starter completed: `ps01/irrigation.py`)

Field (environment): moisture ∈ [0,1], step(m) = min(1, m − heat + rain + irr), heat~U(0.02,0.09), rain∈{0,0.15} w.p. 15%. Weather tapes are **pre-generated per seed and replayed** to both controllers — paired, fair comparison.
SmartIrrigator.perceive: z = true+N(0,0.08); pred = m■–0.055+last; m■ = pred+0.40·(z–pred); attach 80%-accurate binary rain forecast.
decide: effective_target = target – (0.05 if rain forecast else 0); dose = min(max(0, eff–m■)·1.0, 0.16) — *wait-for-rain* discount uses knowledge to skip.
act: clip + apply to Field. **learn**: rolling 14-day stress window; stress → target=min(0.62, +0.012); 14 healthy days → target=max(0.48, –0.008). Initial target 0.55 (safety stock for noise + 20% forecast error).

12-element case-study description (Problem … Ethics)

1. Problem	Keep root-zone moisture above the stress threshold (0.35) for 90 days while minimising pumped water, under noisy sensing and imperfect forecasts.
2. Goal / utility	Minimise Σ irrigation subject to stress-days ≈ 0. Pump-on days reported as energy proxy (many micro-doses vs few floods).
3. Environment	Assiut field: stochastic heat/rain (15% rain days, 0.15 events), soil clipping at 1.0, fully observable only in simulation — partially observable in reality.

4. Sensors / perception	One noisy moisture probe (σ=0.08) + binary rain forecast (80%). Complementary filter (K=0.40) fuses model prediction with reading; forecast is knowledge, not truth.
5. Actuators / action	Dimmable pump: continuous dose [0, 0.16]/day. Timer baseline: 0.12 every 2nd day (45 pump-days, 5.40 water/90d by construction).
6. Knowledge / model	Avg. evaporation 0.055, last dose (process model); forecast semantics ('rain likely → lower effective target by 0.05'); stress threshold 0.35.
7. Decision-making	Deficit rule toward adaptive target with wait-for-rain discount and max-dose cap — a one-step myopic optimiser, transparent and auditable.
8. Learning / adaptation	Parameter-level: target adapts from stress feedback (+0.012 on stress, –0.008 after 14 healthy days, bounded [0.48, 0.62]). No structure learning.
9. Performance measure	Paired 25-seed means: water, stress days, pump-days. Effectiveness = stress ≈ 0; efficiency = water saved; robustness = std across seeds + behaviour under noise/forecast error.
10. Baseline & fairness	Fixed timer on identical weather tapes. Smart: 2.95±0.57 water, 0.24±0.52 stress, 54.8±8.7 pump-days vs timer 5.40±0.00 / 0.00±0.00 / 45.0 — saves 45.4% water at ≈zero stress.
11. Limitations	Forecast is binary & memoryless (no intensity/timing); single soil layer; no pump failures/clogging; evaporation model is a constant mean; 20% forecast error + σ=0.08 still cause ~1 stress day in unlucky seeds (#7: 2 d). Needs field calibration before deployment.
12. Ethics / safety / HITL	Over-irrigation wastes Nile water and leaches fertiliser; under-irrigation threatens livelihood. Design keeps a human gate for large doses (Ex2 pattern), fails safe (never floods on uncertain sensing), logs every dose for audit, and must not disadvantage smallholders if water pricing follows automation.

Evaluation — 90 days × 25 seeds, paired (measured)

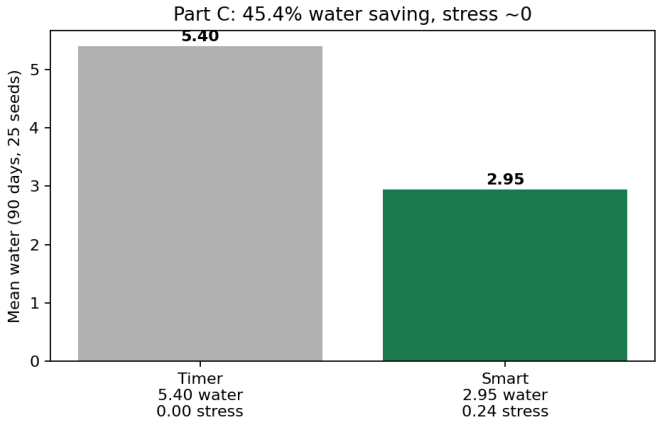


Figure 3 — Mean water: timer 5.40 vs smart 2.95 (–45.4%) with stress 0.00 vs 0.24 d.

Controller	Water (mean±sd)	Stress days (mean±sd)	Pump-days (mean±sd)
Fixed timer	5.40 ± 0.00	0.00 ± 0.00	45.0 ± 0.0
Smart	**2.95 ± 0.57**	**0.24 ± 0.52**	**54.8 ± 8.7**

Table 6 — Summary. Smart saves 2.45 water units (45.4%) at the cost of 0.24 stress days per 90-day season (≈ one stress day every four seasons) — the farmer's 'less water and no stress' goal is met within noise. Pump runs ~10 days more but with micro-doses (avg 0.054/dose vs timer 0.12) — energy per m³ is the next metric to log.

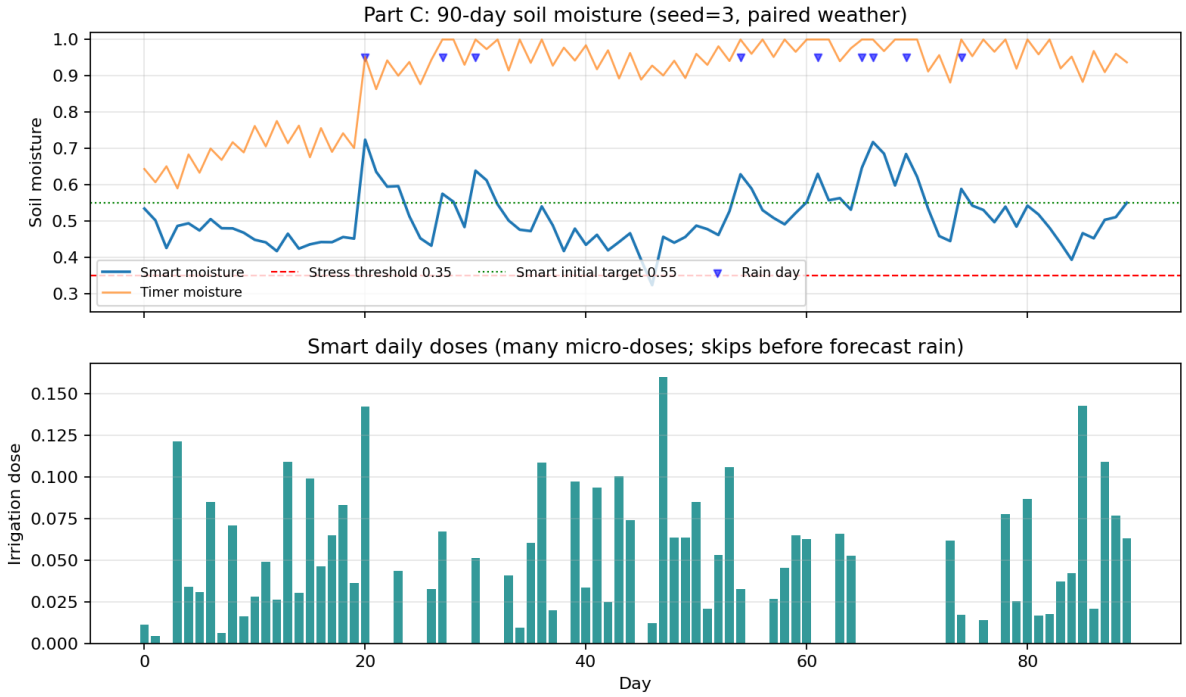


Figure 4 — Paired 90-day trace (seed=3): smart moisture rides just above the target band, doses shrink/skip ahead of (forecast) rain; timer floods on schedule regardless. Red dashed = stress line; blue ticks = true rain days.

Per-seed excerpt (first 8 of 25; full log printed by `irrigation.py`) — water / stress / pump-days:

```
seed 0: timer 5.40/0/45 vs smart 1.88/0/40 (saved +3.52)  •  1: 5.40/0/45 vs 3.77/0/65 (+1.63)  •  2:
5.40/0/45 vs 2.45/0/53 (+2.95)  •  3: 5.40/0/45 vs 3.67/1/64 (+1.73)
4: 5.40/0/45 vs 3.12/1/61 (+2.28)  •  5: 5.40/0/45 vs 3.04/0/57 (+2.36)  •  6: 5.40/0/45 vs 2.12/0/43
(+3.28)  •  7: 5.40/0/45 vs 2.61/2/52 (+2.79) — worst seed still saves 2.79 water.
```

Discussion — why it works; limitations

Three mechanisms stack: (i) **filtering** stops chasing noise (cf. Ex1); (ii) **forecast knowledge** avoids watering before rain (≈15% of days) — the single biggest saver versus the blind timer; (iii) **target adaptation** closes the loop on the true objective (stress) rather than a proxy. The timer's zero stress is purchased with 83% more water — effective but not *efficient*, and in a water-scarce region efficiency *is* effectiveness at scale.

Limitations & next steps: binary forecast ignores amount/timing (a 0.15 event vs drizzle); σ=0.08 and 20% forecast error are assumed known — real deployment needs online estimation; soil is one bucket (no depth, no spatial variation); pump energy is counted in days, not kWh; evaluation is simulation-only. Recommended extensions: probabilistic forecast (expected-rain value), two-layer soil + crop-coefficient, Ex2-style approval gate for doses > 0.12, and a field-calibration week before trusting the 0.55 target.

Appendix — reproducibility & files

All code is commented, seeded, and dependency-light (numpy + matplotlib only for figures). Run from `CSC4103_PS01/ps01/`:

```
python ex1_robust_controller.py   # 60d × 30 runs × 3 σ → Table 1 + Fig.1
python ex2_human_approval.py       # 20 proposals + τ sweep → Tables 2–3
python ex3_autonomy_levels.py     # 22/27/38°C × L0–L3 → Table 5
python irrigation.py                  # 90d × 25 seeds paired → Table 6 + Figs.3–4
python ../make_assets.py              # regenerates assets/*.png
```

Assessment mapping: environment+agent design (§Part C implementation + Fig.2); forecast & noise handling (perceive/decide + Ex1 filter); adaptation from feedback (learn rule + target trace); fair multi-seed evaluation (paired tapes, 25 seeds, mean±sd); 12-element case study + limitations/ethics (numbered list above).

Note on honesty of results: stress is reported as mean±sd over seeds, never a cherry-picked run; the worst smart seed (#7: 2 stress days) is disclosed alongside the mean (0.24). That is the robustness claim — not 'zero stress everywhere' but 'near-zero on average with bounded worst case while saving 45% water'.

End of report — CSC4103 PS1 Parts B+C • Code + figures + this PDF are saved under `Downloads/CSC4103_PS01/`.